

PLANO DE TESTES DE SOFTWARE

Título: Plano de Testes para GitHub

Componentes do Grupo:

- Samuel da Penha Nascimento
 - Paulo Henrique Alves Vieira
 - Gabriel Lima Silva Oliveira
-

1. Introdução e Escolha do Software

1.1 Nome e Versão do Software

O software testado no presente trabalho é o GitHub, plataforma de hospedagem e controle de versão de código, avaliado em sua versão Web (acessível via navegador, sem instalação local).

1.2 Justificativa da Escolha

O GitHub foi selecionado pelos seguintes motivos:

- Possui funcionalidades muito bem definidas e documentadas, facilitando a análise por engenharia reversa;
- É gratuito e de acesso irrestrito, não exigindo licença ou acesso administrativo;
- Apresenta diversas regras de negócio e validações, possibilitando cenários de teste ricos;
- Disponibiliza uma API REST pública e documentada, permitindo a execução de testes funcionais via Postman;
- É amplamente reconhecido pela comunidade acadêmica e pelo mercado, facilitando a contextualização do trabalho;
- Falhas em seus mecanismos de autenticação ou de gerência de código podem comprometer projetos inteiros de software.

1.3 Propósito do Sistema e Público-Alvo

O GitHub é uma plataforma baseada em Git que disponibiliza recursos para controle de versão, colaboração e gerenciamento de repositórios de código-fonte. Entre suas funcionalidades centrais destacam-se: autenticação de usuários, criação e gerenciamento de repositórios públicos e privados, rastreamento de problemas (Issues), revisão de código por Pull Requests e pesquisa global de projetos.

Seus principais usuários são:

- Desenvolvedores individuais e freelancers;
 - Empresas de tecnologia e equipes de engenharia de software;
 - Organizações de código aberto (open source);
 - Estudantes e pesquisadores da área de computação.
-

2. Análise de Requisitos (Engenharia Reversa)

A seguir são descritas as cinco funcionalidades principais identificadas por observação do comportamento do sistema e de sua API REST pública, sem acesso ao código-fonte interno.

- **Funcionalidade 1: Login**
 - **Entrada Esperada:** e-mail do usuário cadastrado e senha da conta;
 - **Saída Esperada:** usuário autenticado com sucesso; dashboard personalizado carregado;
 - **Regras de Negócio:** os campos de e-mail e senha são obrigatórios; o usuário deve estar previamente cadastrado; as credenciais devem coincidir com os dados registrados.
- **Funcionalidade 2: Criar Repositório**
 - **Entrada Esperada:** nome do repositório e definição de visibilidade (público ou privado);
 - **Saída Esperada:** novo repositório criado e visível no perfil do usuário;

- **Regras de Negócio:** o nome é obrigatório e não pode ficar em branco; o nome deve ser único dentro do escopo do usuário autenticado; caracteres especiais inválidos não são permitidos no nome.
 - **Funcionalidade 3: Criar Issue**
 - **Entrada Esperada:** título da issue;
 - **Saída Esperada:** issue registrada e listada na aba Issues do repositório com número sequencial atribuído automaticamente;
 - **Regras de Negócio:** o título é campo obrigatório; o botão de submissão permanece desabilitado enquanto o título estiver vazio; o usuário deve estar autenticado e ter permissão no repositório.
 - **Funcionalidade 4: Criar Pull Request**
 - **Entrada Esperada:** branch de origem (compare) e branch de destino (base);
 - **Saída Esperada:** Pull Request criada com status Open; diferenças entre branches (diff) exibidas para revisão;
 - **Regras de Negócio:** as branches selecionadas devem existir no repositório; deve haver ao menos um commit de diferença entre elas; o sistema deve detectar e alertar sobre conflitos de merge quando presentes.
 - **Funcionalidade 5: Pesquisar Repositórios**
 - **Entrada Esperada:** termo de pesquisa (texto livre);
 - **Saída Esperada:** lista de repositórios correspondentes ao termo pesquisado com filtros disponíveis;
 - **Regras de Negócio:** a pesquisa suporta correspondência parcial; filtros por linguagem, estrelas e data de atualização estão disponíveis; resultados são ordenados por relevância por padrão.
-

3. Estratégia de Testes

A abordagem de testes compreende validações baseadas em técnicas de caixa preta com o auxílio de ferramentas específicas para abranger os espectros funcionais e não funcionais exigidos.

3.1 Teste Funcional (Caixa Preta)

O foco principal é validar a conformidade das regras de negócio do backend e o comportamento esperado da interface gráfica.

- **Postman (API REST):** Utilizado em 9 dos 10 casos de teste para enviar requisições HTTP diretamente aos endpoints da API REST do GitHub (<https://api.github.com>). A autenticação é gerada por meio do cabeçalho Authorization: Bearer SEU_TOKEN. Validam-se os status HTTP retornados (como 200, 201, 401 e 422) e a integridade da resposta em JSON.
- **Execução Manual (UI):** Aplicada complementarmente (como no CT-06) para garantir que restrições do sistema também sejam validadas na camada visual do usuário (bloqueio preventivo de botões).

3.2 Teste de Usabilidade

Avalia a clareza, facilidade de uso e atrito operacional da interface Web. Consiste na observação empírica de critérios de acessibilidade e design de interação, focando em:

- Facilidade de localização de ações críticas (como o botão “New” para criar repositórios ou submissão de issues).
- Responsividade visual da listagem de resultados de pesquisa e legibilidade de telas em resoluções variadas.

3.3 Teste de Performance (Carga/Estresse)

Proposta de avaliação elementar para monitorar o tempo de resposta e o comportamento do sistema sob múltiplas requisições simultâneas. O cenário consiste em utilizar o recurso de automação e coleções do Postman ou scripts via Apache JMeter para disparar requisições concorrentes e repetitivas ao endpoint de pesquisa:

- GET /search/repositories?q=react
- **Métrica avaliada:** Tempo médio de resposta por requisição e estabilidade na entrega do JSON de resposta sem ocorrência de erros HTTP 5xx (Server Error).

3.4 Teste de Segurança (Básico)

Visa validar os controles de autorização, integridade de transmissão e restrições de privilégios de acesso. A validação é concretizada empiricamente por meio de:

- Rejeição explícita de requisições que não utilizem cabeçalhos de autenticação válidos (401 Unauthorized).

- Tentativas de manipulação de parâmetros em endpoints restritos sem tokens com o escopo (*scopes*) de permissão necessário, garantindo que usuários comuns não realizem alterações administrativas.

3.5 Resumo — Ferramenta por Caso de Teste

ID	Funcionalidade	Endpoint / Ação	Resultado Validado	Ferramenta
CT-01	Login	GET /user com token válido	Status 200 + JSON do perfil	Postman
CT-02	Login	GET /user com token inválido	Status 401 Unauthorized	Postman
CT-03	Criar Repositório	POST /user/repos com nome único	Status 201 Created	Postman
CT-04	Criar Repositório	POST /user/repos com nome duplicado	Status 422 Unprocessable Entity	Postman
CT-05	Criar Issue	POST /issues com título válido	Status 201 Created	Postman
CT-06	Criar Issue	POST /issues com título vazio + teste na interface	Status 422 (API) + botão desabilitado (UI)	Postman + Manual
CT-07	Pull Request	POST /pulls com branches distintas	Status 201 Created	Postman
CT-08	Pull Request	POST /pulls com branches idênticas	Status 422 Unprocessable Entity	Postman
CT-09	Pesquisa	GET /search/repositories?q=termo existente	Status 200 + total_count > 0	Postman
CT-10	Pesquisa	GET /search/repositories?q=termo inexistente	Status 200 + total_count = 0	Postman

4. Riscos e Prioridades

A tabela a seguir apresenta o mapeamento de riscos com base na matriz clássica Impacto x Probabilidade, determinando a prioridade de esforço de teste para cada funcionalidade crítica do sistema.

Funcionalidade	Risco / Cenário de Falha	Impacto	Probabilidade	Prioridade
Login	Falha na autenticação / acesso negado a usuário legítimo	Alto	Média	Crítico
Criar Repositório	Dados não persistidos / repositório não criado no servidor	Alto	Baixa	Alto
Criar Issue	Texto da issue perdido / issue não registrada no tracker	Médio	Média	Médio
Criar Pull Request	Merge incorreto / conflito não detectado entre branches	Alto	Baixa	Alto
Pesquisar Repositórios	Nenhum resultado retornado / resultados irrelevantes exibidos	Médio	Alta	Médio

Legenda de prioridade:

- **Crítico:** falha compromete o acesso ao sistema inteiro;
- **Alto:** falha afeta funcionalidade principal com perda de dados;
- **Médio:** falha impacta experiência, mas não impede o uso;
- **Baixo:** falha cosmética ou de baixo impacto operacional.

5. Casos de Teste

Estaseçãoapresentaodetalhamento dos 10 casos de teste projetados para cobrir os principais fluxos lógicos e cenários de exceção.

ID	Funcionalidade	Cenário	Pré-condição	Passos de Execução	Dados de Entrada	Resultado Esperado
CT-01	Login	Login com sucesso	Usuário cadastrado e não autenticado	1. Enviar GET /user no Postman com token válido. 2. Verificar status HTTP da resposta. 3. Conferir retorno dos dados do perfil.	Authorization: Bearer TOKEN_VALIDO	Status 200 OK; JSON com dados do perfil retornado.
CT-02	Login	Login com credenciais inválidas	Usuário não autenticado	1. Enviar GET /user no Postman com token inválido. 2. Verificar status HTTP da resposta. 3. Conferir mensagem de erro retornada.	Authorization: Bearer TOKEN_INVALIDO	Status 401 Unauthorized mensagem d erro no JSON
CT-03	Criar Repositório	Criação com nome válido e único	Usuário autenticado com token válido	1. Enviar POST /user/repos no Postman. 2. Inserir body JSON com nome único. 3. Verificar status e resposta.	{ "name": "repo-teste-2026", "private": false }	Status 201 Created; JSON com dados do repositório criado.
CT-04	Criar Repositório	Nome de repositório duplicado	Usuário autenticado; repositório 'repo-teste-2026' já existe	1. Enviar POST /user/repos no Postman. 2. Inserir o mesmo nome já existente. 3. Verificar status e mensagem de erro.	{ "name": "repo-teste-2026", "private": false }	Status 422 Unprocessab Entity; mensagem indicando nome já utilizado.
CT-05	Criar Issue	Issue criada com título válido	Usuário autenticado; repositório existente com Issues habilitadas	1. Enviar POST /repos/{owner}/{repo}/issues no Postman. 2. Inserir body com título válido. 3. Verificar status e número da issue.	{ "title": "Bug na autenticacao SSO" }	Status 201 Created; JSON com número sequencial da issue.
CT-06	Criar Issue	Tentativa de criar issue sem título	Usuário autenticado; repositório existente com Issues habilitadas	API: 1. Enviar POST /repos/{owner}/{repo}/issues com titulo vazio. 2. Verificar status. UI: 1. Acessar aba Issues. 2. Clicar em New issue. 3. Deixar titulo em branco. 4. Tentar submeter.	API: { "title": "" } UI: titulo vazio	API: Status 422 Unprocessab Entity. UI: botao Submit desabilitado.
CT-07	Pull Request	PR criada com branches válidas e com diferenças	Usuário autenticado; branches 'main' e 'feature/login' com commits distintos	1. Enviar POST /repos/{owner}/{repo}/pulls no Postman. 2. Inserir branches de origem e destino. 3. Verificar status e diff retornado.	{ "title": "PR de teste", "head": "feature/login", "base": "main" }	Status 201 Created; PR com status Open e diff exibido.
CT-08	Pull Request	PR com branches sem diferenças	Usuário autenticado; branches 'main' e	1. Enviar POST /repos/{owner}/{repo}/pulls no Postman. 2. Selecionar branches	{ "title": "PR vazia", "head": "copia-main", "base": "main" }	Status 422 Unprocessab Entity; mensagem indicando

ID	Funcionalidade	Cenário	Pré-condição	Passos de Execução	Dados de Entrada	Resultado Esperado
			'copia-main' idênticas	idênticas. 3. Verificar resposta da API.		ausência de diferenças.
CT-09	Pesquisa	Pesquisa por repositório existente	Usuário autenticado	1. Enviar GET /search/repositories? q=facebook/react no Postman. 2. Verificar status. 3. Conferir total_count e itens retornados.	q=facebook/react	Status 200 OK; total_count > 0; repositório facebook/react na lista.
CT-10	Pesquisa	Pesquisa por termo inexistente	Usuário autenticado	1. Enviar GET /search/repositories? q=xyzxyz_naoexiste_abc123 no Postman. 2. Verificar status. 3. Conferir total_count e lista de itens.	q=xyzxyz_naoexiste_abc123	Status 200 OK; total_count = 0; lista de itens vazia.

6. Cronograma e Alocação

6.1 Estimativa de Tempo de Execução

A execução completa do ciclo de testes foi estimada em 4 dias úteis:

- **Dia 1 — Engenharia Reversa e Planejamento:** levantamento de funcionalidades, definição de escopo e elaboração dos casos de teste (estimativa: 4 horas);
- **Dia 2 — Configuração do Postman e execução dos casos CT-01 a CT-06:** geração do token, configuração dos endpoints e registro dos resultados (estimativa: 3 horas);
- **Dia 3 — Execução dos casos CT-07 a CT-10 e verificação manual do CT-06:** conclusão dos testes e registro dos resultados (estimativa: 2 horas);
- **Dia 4 — Consolidação e Relatório Final:** registro dos defeitos encontrados, redação da conclusão e preparação da apresentação (estimativa: 3 horas).

6.2 Divisão de Responsabilidades

- **Samuel da Penha Nascimento:** Seções 1 e 2 (Introdução e Engenharia Reversa); execução dos casos de teste CT-01, CT-02, CT-05 e CT-06 via Postman e verificação manual; registro de defeitos identificados.
- **Paulo Henrique Alves Vieira:** Seções 3 e 4 (Estratégia de Testes e Matriz de Riscos); execução dos casos de teste CT-03, CT-04, CT-07 e CT-08 via Postman; formatação e revisão final.
- **Gabriel Lima Silva Oliveira:** Seções 6 e 7 (Cronograma e Conclusão); execução dos casos de teste CT-09 e CT-10 via Postman; elaboração da tabela de riscos e critérios de aceitação.

7. Conclusão e Critérios de Aceitação

7.1 Critérios de Entrada e Saída

Baseando-se em diretrizes normativas de qualidade de software, estabelecemos as seguintes condições reguladoras:

- **Critérios de Entrada:** Código do software disponível em ambiente estável (versão Web), massa de dados para pesquisa inicial pronta, Postman instalado localmente e *Personal Access Token (PAT)* gerado com escopos apropriados de leitura e gravação.
- **Critérios de Saída:** 100% dos casos de teste de prioridade Crítica e Alta executados com resultado de aprovação; ausência de bugs severos bloqueantes; e taxa de sucesso geral nos testes funcionais igual ou superior a 90%.

7.2 Definição de Software Aprovado

O GitHub será considerado aprovado neste ciclo de testes caso atenda estritamente às métricas descritas nos critérios de saída, mantendo a integridade dos endpoints avaliados, a estabilidade de resposta abaixo de limites de tolerância de latência e nenhuma quebra evidente nas regras de validação lógica de dados de entrada.